Report 4: Word-Level Confidence Scoring

Document Classification: Technical Research Report **Date:** February 17, 2026 **Scope:** How to extract, compute, and use per-word confidence scores from VSP-LLM **Current State:** Model outputs text only — no scores, probabilities, or confidence indicators are captured

---

1. Executive Summary

The VSP-LLM pipeline currently discards **all confidence information** during decoding. The model performs beam search with 20 candidates, selects the best hypothesis, and saves only the text. No scores, probabilities, or alternative hypotheses are preserved.

This is the single most impactful engineering gap in the current system. If we exposed confidence scores, we could: 1. **Flag low-confidence words** for human review (instead of reviewing everything) 2. **Estimate segment-level reliability** without ground truth (solving the "can we separate good from bad?" problem) 3. **Color-code words** in reports by confidence (green/yellow/red) 4. **Set automatic rejection thresholds** — discard segments below a confidence floor

This report details exactly what confidence information is available inside the model, how to extract it, and how to surface it in the reporting pipeline.

---

2. What Confidence Information Exists (But Is Discarded)

2.1 The HuggingFace `generate()` API

The model calls `self.decoder.generate()` (LLaMA-2) which supports several output modes:

```
# Current call (vsp_llm.py, lines 386-396):
outputs = self.decoder.generate(
    inputs_embeds=llm_input,
    num_beams=num_beams,
    max_new_tokens=max_length,
    ...
)
# Returns: tensor of token IDs only (shape: [batch, seq_len])
```

```
# Enhanced call (what we SHOULD do):
outputs = self.decoder.generate(
    inputs_embeds=llm_input,
```

```
    num_beams=num_beams,
    max_new_tokens=max_length,
    output_scores=True,                  # NEW: return per-step scores
    return_dict_in_generate=True,        # NEW: return structured output
    ...
)
# Returns: GenerateBeamDecoderOnlyOutput with:
#   .sequences:      [batch, seq_len] — token IDs (same as before)
#   .sequences_scores: [batch] — log-probability of each complete sequence
#   .scores:         tuple of [batch*num_beams, vocab_size] at each step
#   .beam_indices:   [batch, seq_len] — which beam each token came from
```

2.2 Available Confidence Signals

Signal	Granularity	Description	Currently Captured
Sequence score	Per-sequence	Log-probability of the complete hypothesis	No
Token scores	Per-token	Raw logits over full vocabulary at each generation step	No
Token probabilities	Per-token	Softmax of token scores — probability of each token	No
Beam indices	Per-token	Which beam each token was selected from	No
Top-k alternatives	Per-token	Most likely alternative tokens at each position	No
Sequence alternatives	Per-sequence	The other 19 beam search candidates	No

2.3 What Each Signal Tells Us

Sequence score — A single number representing how confident the model is in the entire hypothesis. Higher (less negative) = more confident. Useful for segment-level quality estimation.

Token probabilities — At each generation step, the model produces a probability distribution over its entire vocabulary (~32,000 tokens for LLaMA-2). The probability assigned to the selected token indicates confidence. If the model assigns 0.95 to a token, it's very confident. If it assigns 0.05, it's essentially guessing.

Entropy — The entropy of the probability distribution at each step. Low entropy = model is certain (one token dominates). High entropy = model is uncertain (probability spread across many tokens).

---

3. How to Extract Token-Level Confidence

3.1 Code Modification: `vsp_llm.py`

The `generate()` method needs to return structured output:

```
# File: VSP-LLM/src/vsp_llm.py
# Replace lines 386-398:

@torch.no_grad()
def generate(self,
             num_beams=20,
             max_length=30,
             min_length=1,
             top_p=0.9,
             repetition_penalty=1.0,
             length_penalty=0.0,
             no_repeat_ngram_size=0,
             do_sample=False,
             temperature=1.0,
             return_confidence=False,   # NEW parameter
             **kwargs,
             ):
    output = self.encoder(**kwargs)
    output['encoder_out'] = self.avfeat_to_llm(output['encoder_out'])
    cluster_counts = kwargs['source']['cluster_counts'][0]

    results_tensor = []
    start_idx = 0
    for clutser_num in cluster_counts:
        end_idx = start_idx + clutser_num
        slice = output['encoder_out'][:,start_idx:end_idx,:]
        mean_tensor = torch.mean(slice, dim=1, keepdim=True)
        results_tensor.append(mean_tensor)
        start_idx = end_idx

    assert(cluster_counts.sum().item() == output['encoder_out'].size()[1])
    reduced_enc_out = torch.cat(results_tensor, dim=1)
    B, T, D = reduced_enc_out.size()
    instruction = kwargs['source']['text']
    instruction_embedding = self.decoder.model.model.embed_tokens(instruction)
    llm_input = torch.cat((instruction_embedding, reduced_enc_out), dim=1)

    self.decoder.config.use_cache = True

    if return_confidence:
        gen_output = self.decoder.generate(
            inputs_embeds=llm_input,
            top_p=top_p,
            num_beams=num_beams,
            max_new_tokens=max_length,
            min_length=min_length,
```

```
            repetition_penalty=repetition_penalty,
            do_sample=do_sample,
            temperature=temperature,
            length_penalty=length_penalty,
            no_repeat_ngram_size=no_repeat_ngram_size,
            output_scores=True,
            return_dict_in_generate=True,
        )
        return gen_output  # Returns GenerateBeamDecoderOnlyOutput
    else:
        outputs = self.decoder.generate(
            inputs_embeds=llm_input,
            top_p=top_p,
            num_beams=num_beams,
            max_new_tokens=max_length,
            min_length=min_length,
            repetition_penalty=repetition_penalty,
            do_sample=do_sample,
            temperature=temperature,
            length_penalty=length_penalty,
            no_repeat_ngram_size=no_repeat_ngram_size,
        )
        return outputs  # Backward compatible
```

3.2 Code Modification: `vsp_llm_decode.py`

Extract confidence from the structured output:

```
# File: VSP-LLM/src/vsp_llm_decode.py
# After model.generate() call (around line 238):

import torch.nn.functional as F
import math

gen_output = model.generate(
    target_list=sample["target"],
    num_beams=cfg.generation.beam,
    max_length=dynamic_max_len,
    length_penalty=cfg.generation.lenpen,
    no_repeat_ngram_size=cfg.generation.no_repeat_ngram_size,
    repetition_penalty=cfg.generation.repetition_penalty,
    do_sample=cfg.generation.do_sample,
    temperature=cfg.generation.temperature,
    top_p=cfg.generation.top_p,
    return_confidence=True,           # NEW
    **sample["net_input"]
)

# Extract sequences (same as before)
best_hypo_tokens = gen_output.sequences

# Extract sequence-level score
```

```
seq_scores = gen_output.sequences_scores  # [batch_size]

# Extract per-token confidence
token_confidences = []
token_entropies = []
token_alternatives = []

for step_scores in gen_output.scores:
    # step_scores shape: [batch_size * num_beams, vocab_size]
    # We want the scores for the best beam only
    # After beam search, take every num_beams-th entry (beam 0)
    best_beam_scores = step_scores[::cfg.generation.beam]  # [batch_size, vocab_size]

    # Convert to probabilities
    probs = F.softmax(best_beam_scores, dim=-1)

    # Token confidence = probability of the selected token
    selected_token_ids = best_hypo_tokens[:, len(token_confidences) + 1]  # +1 for BOS
    confidence = probs.gather(1, selected_token_ids.unsqueeze(1)).squeeze(1)
    token_confidences.append(confidence.cpu().tolist())

    # Entropy = uncertainty measure
    entropy = -(probs * probs.log()).sum(dim=-1)
    token_entropies.append(entropy.cpu().tolist())

    # Top-3 alternatives
    top3_probs, top3_ids = probs.topk(3, dim=-1)
    token_alternatives.append({
        'ids': top3_ids.cpu().tolist(),
        'probs': top3_probs.cpu().tolist()
    })

# Reshape: [steps, batch] -> [batch, steps]
token_confidences = list(zip(*token_confidences))
token_entropies = list(zip(*token_entropies))
```

3.3 Mapping Token Confidence to Word Confidence

LLaMA-2 uses a BPE (Byte-Pair Encoding) tokenizer, meaning words can be split into multiple tokens: - "cartilages" → ["cart", "il", "ages"] (3 tokens) - "the" → ["the"] (1 token)

To get word-level confidence:

```
def token_to_word_confidence(tokens, confidences, tokenizer):
    """Aggregate sub-token confidences into word-level scores."""
    text = tokenizer.decode(tokens, skip_special_tokens=True)
    words = text.split()

    word_confidences = []
    token_idx = 0
    for word in words:
        # Find which tokens compose this word
```

```
        word_tokens = tokenizer.encode(word, add_special_tokens=False)
        n_tokens = len(word_tokens)

        # Aggregate: use minimum confidence across sub-tokens
        # (weakest link — if any sub-token is uncertain, the word is uncertain)
        word_conf_values = confidences[token_idx:token_idx + n_tokens]
        if word_conf_values:
            word_confidence = min(word_conf_values)  # Conservative
            # Alternative: geometric mean
            # word_confidence = math.exp(sum(math.log(c) for c in word_conf_values) / len(w
        else:
            word_confidence = 0.0

        word_confidences.append({
            'word': word,
            'confidence': word_confidence,
            'n_tokens': n_tokens
        })
        token_idx += n_tokens

    return word_confidences
```

Aggregation strategies for multi-token words:

Strategy	Formula	Best For
Minimum (recommended)	`min(token_probs)`	Conservative — flags uncertain words
Geometric mean	`exp(mean(log(probs)))`	Balanced — averages uncertainty
Arithmetic mean	`mean(probs)`	Optimistic — may miss low-confidence sub-tokens
First token	`first_token_prob`	Fast — often the most informative token

---

4. Confidence-Based Quality Estimation

4.1 Segment-Level Confidence Score

```
def segment_confidence(word_confidences):
    """Compute overall segment confidence from word-level scores."""
    if not word_confidences:
        return 0.0

    confs = [w['confidence'] for w in word_confidences]

    return {
```

```
        'mean_confidence': sum(confs) / len(confs),
        'min_confidence': min(confs),
        'median_confidence': sorted(confs)[len(confs)//2],
        'low_confidence_ratio': sum(1 for c in confs if c < 0.3) / len(confs),
        'geometric_mean': math.exp(sum(math.log(max(c, 1e-10)) for c in confs) / len(confs)
    }
```

4.2 Expected Confidence Distributions

Based on the model's behavior:

Segment Quality	Expected Mean Confidence	Expected Min Confidence
Perfect (WER=0)	> 0.7	> 0.3
Good (WER≤20%)	0.5 - 0.8	0.1 - 0.4
Poor (WER>60%)	0.2 - 0.5	< 0.1
Hallucinated (WER≥100%)	0.1 - 0.4 (may be high!)	< 0.05

Important caveat: Hallucinated text may have **high confidence** because the LLM is confidently generating fluent text from its language prior. The model may be "confident" in its hallucination because it's essentially doing language modeling rather than lip reading. This is a known problem with neural model calibration (Guo et al., 2017).

4.3 Entropy as Uncertainty Signal

Entropy captures how "spread out" the probability distribution is: - **Low entropy (< 1.0):** Model is very certain — one token dominates - **Medium entropy (1.0 - 3.0):** Moderate uncertainty — a few strong candidates - **High entropy (> 3.0):** High uncertainty — many plausible tokens

Entropy may be a **better** indicator of hallucination than raw confidence because: - A hallucinating model may assign high probability to its chosen (wrong) token - But the entropy will be low only if the model truly has strong visual evidence - When hallucinating from language prior, the model often has medium entropy (several plausible continuations)

```
def compute_entropy(logits):
    """Compute entropy of the probability distribution."""
    probs = F.softmax(logits, dim=-1)
    log_probs = F.log_softmax(logits, dim=-1)
    entropy = -(probs * log_probs).sum(dim=-1)
    return entropy.item()
```

---

5. Integrating Confidence into Reports

5.1 Enhanced Report Format

Modify `make_report.py` to include confidence scoring:

```
# New CSV columns:
# utt_id, display_name, ref, hyp, hyp_tagged, wer_%, wwer_%,
# nea_recall_%, nea_precision_%, nea_f1_%, missed_entities,
# seq_confidence, mean_word_confidence, min_word_confidence,   # NEW
# low_confidence_ratio, mean_entropy, hyp_with_confidence      # NEW
```

5.2 Color-Coded Confidence in HTML Report

```
<!-- Word colored by confidence -->
<span style="background-color: rgba(0, 255, 0, 0.7);">high-conf-word</span>
<span style="background-color: rgba(255, 255, 0, 0.7);">medium-conf-word</span>
<span style="background-color: rgba(255, 0, 0, 0.7);">low-conf-word</span>
```

Confidence thresholds: | Confidence | Color | Interpretation | |----------|-------|---------------| | ≥ 0.7 | Green | Likely correct | | 0.3 - 0.7 | Yellow | Uncertain — review recommended | | < 0.3 | Red | Likely wrong |

5.3 Confidence-Based Filtering

```
# Automatic quality tiers based on confidence
def classify_segment(seq_confidence, mean_word_conf, low_conf_ratio):
    if mean_word_conf >= 0.6 and low_conf_ratio < 0.2:
        return "HIGH_CONFIDENCE"  # Likely trustworthy
    elif mean_word_conf >= 0.3 and low_conf_ratio < 0.5:
        return "MEDIUM_CONFIDENCE"  # Review recommended
    else:
        return "LOW_CONFIDENCE"  # Do not trust
```

---

6. Advanced: Calibrating Confidence Scores

6.1 The Calibration Problem

Neural network confidence scores are often **poorly calibrated** — the model may output 0.9 probability for a word that is correct only 60% of the time, or 0.3 for a word that is correct 80% of the time (Guo et al., 2017, "On Calibration of Modern Neural Networks").

6.2 Temperature Scaling

A simple post-hoc calibration technique:

```
# After obtaining logits, apply a learned temperature
calibrated_probs = F.softmax(logits / T_calibration, dim=-1)
# T_calibration > 1.0: softens probabilities (reduces overconfidence)
# T_calibration < 1.0: sharpens probabilities
```

To learn `T_calibration`: 1. Run the model on a held-out set with ground truth 2. Find `T` that minimizes negative log-likelihood of the correct tokens 3. Apply this `T` at inference time

6.3 Platt Scaling

For segment-level confidence: 1. Extract raw sequence scores for a development set 2. Fit a logistic regression: `P(correct) = sigmoid(a * raw_score + b)` 3. Use the fitted `a, b` to convert raw scores to calibrated probabilities

---

7. Memory and Performance Implications

7.1 Memory Cost of `output_scores=True`

With `output_scores=True`, the model stores the full vocabulary distribution at each step: - Vocabulary size: ~32,000 tokens - Each step: `batch_size * num_beams * 32,000 * 4 bytes` (float32) - For beam=20, batch=1: `1 * 20 * 32,000 * 4 = 2.56 MB per step` - For 200 steps: `512 MB total`

This is significant but manageable. To reduce memory: - Process scores step-by-step instead of storing all at once - Reduce beam width to 5 (reduces memory 4x) - Only store top-k scores per step (e.g., top 100 instead of full 32,000)

7.2 Compute Overhead

The confidence extraction itself is lightweight — just softmax and gather operations. The overhead comes from: 1. **Storing scores during generation:** ~10-15% slowdown 2. **Post-processing (softmax, entropy):** Negligible 3. **Serialization to JSON:** Additional ~20% I/O time for the larger output files

Recommendation: Enable confidence scoring by default. The 10-15% decode slowdown is acceptable given the dramatic improvement in output usability.

---

8. What Confidence Scoring Enables (the payoff)

8.1 Answering "Can We Separate Good from Bad?"

Report 1 showed that no production-available heuristic could reliably separate good from bad predictions (best precision: 24%). Confidence scores change this equation entirely:

Signal	Pearson r with WER (expected)	Source
Hypothesis word count	-0.166	Available today
Segment duration	-0.151	Available today
Sequence score	**-0.4 to -0.6** (estimated)	Requires code change
Mean word confidence	**-0.3 to -0.5** (estimated)	Requires code change
Low-confidence word ratio	**+0.3 to +0.5** (estimated)	Requires code change
Mean entropy	**+0.2 to +0.4** (estimated)	Requires code change

These estimates are based on ASR confidence literature (Jiang, 2005; Seigel & Woodland, 2011; Ragni et al., 2018). Even a modest r = 0.4 correlation would enable filtering with ~40-50% precision at 60% recall — roughly **doubling** the current best heuristic.

8.2 Practical Workflow

Without confidence (current):

```
860 segments → Human reviews ALL 860 → finds 98 good ones → hours wasted
```

With confidence scoring:

```
860 segments → Sort by confidence → Review top 200 → find ~80 good ones → 5x faster
```

8.3 Per-Word Confidence for Partial Trust

Even in segments with WER ~50%, many individual words are correct. Confidence scoring enables:

```
HYP: "the speaker is discussing [0.85] anatomy [0.15] of the [0.72] human [0.65] body [0.88
                                                    ^^^^^^^
                                          This word is likely wrong — flag for review
```

The user gets actionable intelligence: "most of this sentence is probably right, but 'anatomy' is uncertain."

---

9. Implementation Roadmap

Phase 1: Sequence-Level Score (2-4 hours)

1. Modify `vsp_llm.py` generate() to accept `return_confidence=True`
2. Add `output_scores=True, return_dict_in_generate=True` to decoder.generate()
3. Extract `sequences_scores` — a single number per hypothesis
4. Save in decode output JSON alongside hypo text
5. Add to make_report.py as a new column

This alone is sufficient to answer the separation question. Run the model once with scores, then correlate `sequences_scores` with known WER to validate.

Phase 2: Token-Level Confidence (1-2 days)

1. Extract per-step scores from `gen_output.scores`
2. Compute token-level probabilities and entropy
3. Map tokens to words using tokenizer alignment
4. Save word-level confidence in decode output JSON
5. Update make_report.py for confidence-colored HTML output

Phase 3: Calibration (3-5 days)

1. Run model on development set with ground truth
2. Fit temperature scaling or Platt scaling
3. Validate calibration on held-out test set
4. Apply calibration to production inference

---

10. References

1. Yeo et al. (2024). "VSP-LLM." arXiv:2402.15151.
2. Guo et al. (2017). "On Calibration of Modern Neural Networks." ICML 2017. — Neural network calibration.
3. Jiang (2005). "Confidence Measures for Speech Recognition: A Survey." Speech Communication. — Comprehensive ASR confidence review.
4. Seigel & Woodland (2011). "Combining Information Sources for Confidence Estimation with CRF Models." Interspeech 2011.

5. Ragni et al. (2018). "Confidence Estimation and Deletion Prediction Using Bidirectional Recurrent Neural Networks." IEEE SLT 2018.
6. Platt (1999). "Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods." — Platt scaling.
7. HuggingFace Transformers Documentation. "Generation with LLMs — output_scores parameter." — API reference for GenerateOutput.
8. Malinin & Gales (2021). "Uncertainty Estimation in Autoregressive Structured Prediction." ICLR 2021. — Sequence-level uncertainty.